

Lab 1 Linear Regression (Resubmission)

MACHINE LEARNING FOR THE BUILT ENVIRONMENT

GEO5017

Authors:

Xinya Bi (6195350)
Hongyu Ye (6286240)
Xu Wang (6235379)

March 17, 2025



Contents

Notes for the Resubmission	1
1 Question 1	2
2 Question 2	2
2.1 Trajectory Plotting	2
2.2 Polynomial Regression	2
2.2.1 Constant Speed Model	2
2.2.2 Constant Acceleration Model	2
2.2.3 Location Prediction	3
3 Work Allocation	4
4 Additional Graphs and Data	4

Notes for the Resubmission

All differences from the original report are marked in [blue font](#), and for each point in the teacher's feedback we made the following changes:

1. **1.1** "plot the data" and understand the relationships" is not sufficient. One should also hypothesize the model. (-5)
Added the hypothesis and description of the model.
2. **1.2** "Parameters can minimize..." → "The optimal solution is the set of parameters that minimize..." (-2)
Revised the content to ensure a more rigorous statement.
3. **2.2 (b)** Your report doesn't show the final models for the constant acceleration case. (-5)
Moved the final fitted functions from the original Section 2.2.2(3) "improvement" part to the results presentation section.
4. **2.2 (b)** Your code `3_constant_acceleration_model.py` doesn't produce the expected models or the model parameters. (-5)
Removed the original `3_constant_acceleration_model.py`, because `4_constant_acceleration_model_improved.py` already serves as the final Constant Acceleration Model code, which prints the expected models or model parameters.
5. **2.2 (c)** You start with a relatively large learning rate and then gradually reduce it. This is a good strategy for improving gradient descent. But the question asks you to improve your "model" (not the solver). (-10)
We introduced a cubic polynomial model to properly improve the model.

1 Question 1

- (1) We need to plot the data points to explore the possible relationships between independent and dependent variables.
- (2) The optimal solution is the set of parameters that minimize the sum of squared errors between predicted values and observed values.

2 Question 2

2.1 Trajectory Plotting

We use matplotlib to plot the trajectory of the drone, see Figure 1. According to the plotted results, we observe that in the x , y , and z directions, the position can be approximately fitted with a cubic model with respect to t :

$$y = a_0 + a_1x + a_2x^2 + a_3x^3$$

2.2 Polynomial Regression

To solve the time and position problem, we first design a gradient descent algorithm that can handle simple and polynomial linear regression models (Keep the first 3 digits after the decimal point).

1. Starting with an initial parameter guess.
2. In each iteration
 - (a) calculating the step size by multiplying learning rate and gradient. Gradient is the derivative of the loss function with respect to every unknown parameters.
 - (b) Checking if all elements of the step size are smaller than the threshold. If so, the algorithm has converged and exits early.
 - (c) Updating the new parameters by subtracting the step size from the current parameters.

2.2.1 Constant Speed Model

Best Parameters (Shown in Figure 2): $Learning\ Rate = 0.01$, $Max\ Iterations = 200$, $threshold = 0.001$

$$\begin{array}{ll} x_{pos} = 1.424 - 0.431 \cdot t & SSE: 8.330 \\ y_{pos} = 2.081 - 0.580 \cdot t & SSE: 6.024 \\ z_{pos} = 0.754 + 0.493 \cdot t & SSE: 1.634 \end{array}$$

After testing different parameter values (learning rates, iteration counts, residuals), we found minimal differences in results. We decided the best parameters for the model are $Learning\ Rate = 0.01$, $Max\ Iterations = 200$, $threshold = 0.001$. It has a few advantages: (1) It has the lowest residual errors across 3 dimensions. (2) It converges after about 150 iterations. The speed is $x_v = -0.431$, $y_v = -0.580$, $z_v = 0.493$

2.2.2 Constant Acceleration Model

(1) Methods

(A) Establishing the Model

Assuming constant acceleration, the motion of a drone in 3-dimensional space can be described by the following second-order polynomial equations in the x , y , and z directions:

$$\begin{cases} x(t) = a_x + b_x t + c_x t^2, \\ y(t) = a_y + b_y t + c_y t^2, \\ z(t) = a_z + b_z t + c_z t^2. \end{cases}$$

Here, (a_x, b_x, c_x) correspond to the coefficients in the x direction. The same applies to (a_y, b_y, c_y) and (a_z, b_z, c_z) .

(B) Optimization of models

A major drawback of the present solver is selecting an appropriate learning rate is needed, as small variations in α can drastically impact results, making manual tuning inefficient.

To address this, we implemented a decaying learning rate, starting with a relatively larger α that gradually decreases during gradient descent. Our approach halves α at intervals, eliminating manual tuning while maintaining effective convergence (Figure 4).

In addition, by observing the velocity dependence of the metadata, and the heading angle in the x-y plane and its changes (Figure 5), we consider taking into account the angular velocity and the covariance constraints of x-y-z when fitting the model. Considering the complexity of calculation and operation, we use PyTorch to optimize the process.

(2) Results

Our fitted function yields the following fitted equations and corresponding fitting performance (Figure 6 and Figure 9):

$$\begin{aligned}x(t) &= 5.516000 - 3.47714 \cdot t + 0.433571 \cdot t^2 & \text{SSE: } 0.655 \\y(t) &= -0.894014 + 1.674696 \cdot t - 0.323591 \cdot t^2 & \text{SSE: } 1.057 \\z(t) &= 1.311000 + 0.098822 \cdot t + 0.054821 \cdot t^2 & \text{SSE: } 0.760\end{aligned}$$

We set the maximum iterations to 6×10^4 and consider convergence achieved when the step size falls below 10^{-6} . The coefficients start from zero. Under these settings, we tested different learning rates α (Figure 3).

If α is too small, convergence is slow and computationally expensive; if too large, it may fail to converge. To optimize efficiency, we analyze the cost function and select the largest α before divergence (Table 1).

Our algorithm achieved a minimum SSE of 0.655 on x , 1.057 on y , and 0.760 on z . significantly outperforming the Constant Speed Model due to the superior fitting capability of a quadratic polynomial.

In addition, the results of the training simulation using the angular velocity and xyz covariance constraints show that the total SSE is optimized by about only 10^{-5} (2.471007 to 2.470989) compared to only applying the decaying learning rate. However, after applying covariance and angular velocity regularization, the model is more constrained and the optimization process is more stable and faster, thus reducing the number of iterations (Figure 8).

(3) Improvement

In order to achieve better fitting performance, we use a cubic model:

$$\begin{cases} x(t) = a_x + b_x t + c_x t^2 + d_x t^3, \\ y(t) = a_y + b_y t + c_y t^2 + d_y t^3, \\ z(t) = a_z + b_z t + c_z t^2 + d_z t^3. \end{cases}$$

Using the cubic polynomial model, as shown in Figure 10, we obtain the following fitted functions and SSE values:

$$\begin{aligned}x(t) &= 1.783415 + 1.130270 \cdot t - 1.076257 \cdot t^2 + 0.142846 \cdot t^3 & \text{SSE: } 0.107 \\y(t) &= -4.300199 + 6.069600 \cdot t - 1.802661 \cdot t^2 + 0.142346 \cdot t^3 & \text{SSE: } 0.101 \\z(t) &= -1.523532 + 3.774587 \cdot t - 1.186074 \cdot t^2 + 0.119661 \cdot t^3 & \text{SSE: } 0.045\end{aligned}$$

We can see that for the x , y , and z directions, the SSE is significantly reduced, and the fitting performance for local trend variations is notably improved (Figure 11).

2.2.3 Location Prediction

Using Constant Acceleration Model, the predicted coordinates at $t = 7$ are: $x = 2.424$, $y = -5.027$, $z = 4.689$. The position prediction results of the fitted function are shown in Figure 7.

3 Work Allocation

- Xinya Bi (40%): gradient descent algorithm, trajectory plotting, constant speed model
- Hongyu Ye (30%): Implementation of gradient descent algorithms with improvements, constant acceleration models and improved trajectory plotting.
- Xu Wang (30%): Implementation of gradient descent algorithms with improvements in the angular velocity and xyz covariance constraints.

4 Additional Graphs and Data

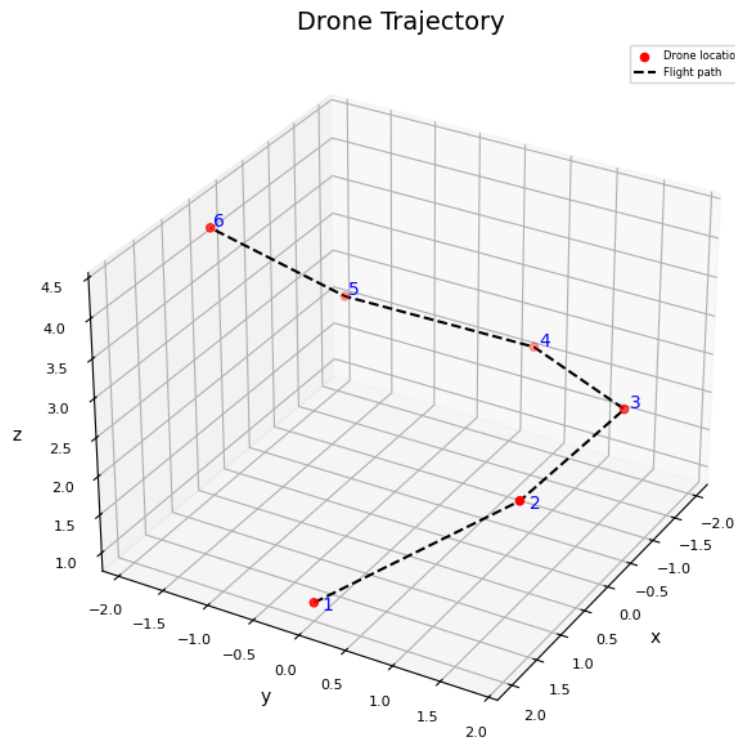


Figure 1: Drone Trajectory

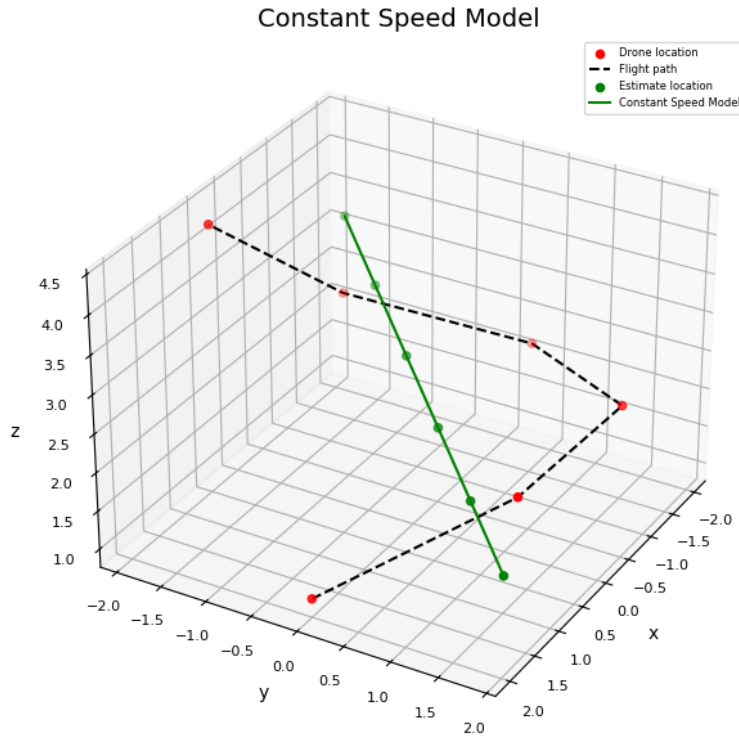


Figure 2: Constant Speed Model and the Actual Position

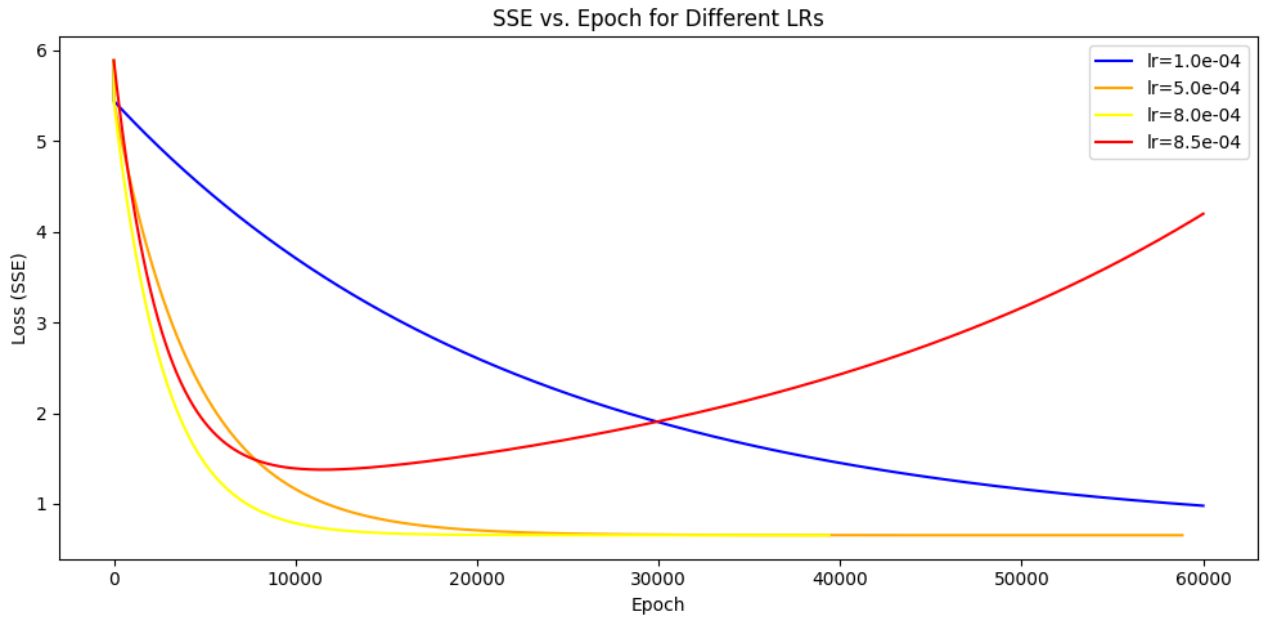


Figure 3: SSE and Epoch for Different LR's on x

Table 1: Gradient Descent Results with Different LR's on x

Learning Rate	Epochs	Step Size	Final SSE
1.00×10^{-4}	60000	3.82×10^{-5}	0.980
5.00×10^{-4}	58847	1.00×10^{-6}	0.655
8.00×10^{-4}	39399	1.00×10^{-6}	0.655
8.46×10^{-4}	60000	1.10×10^{-1}	4.199

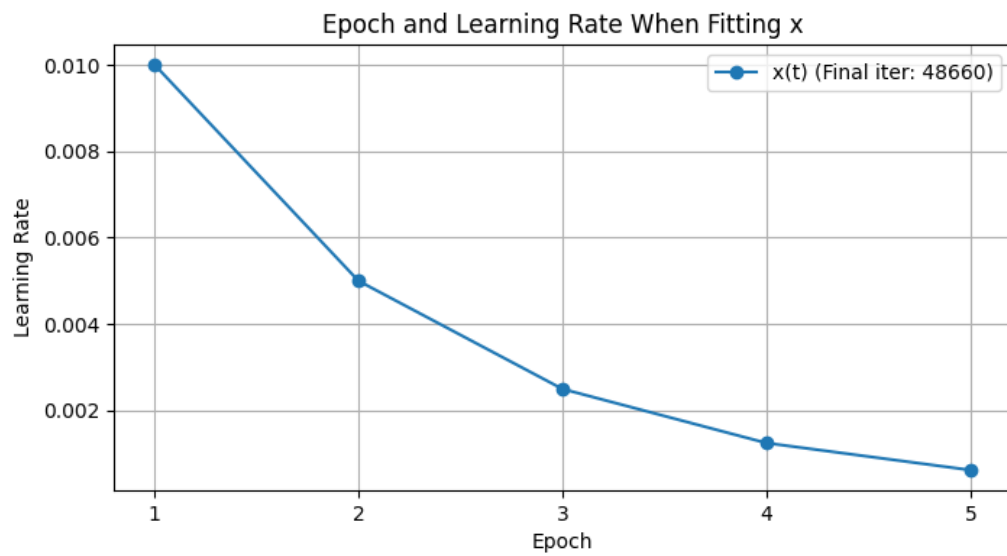


Figure 4: Decaying Learning Rate on x-position Fitting

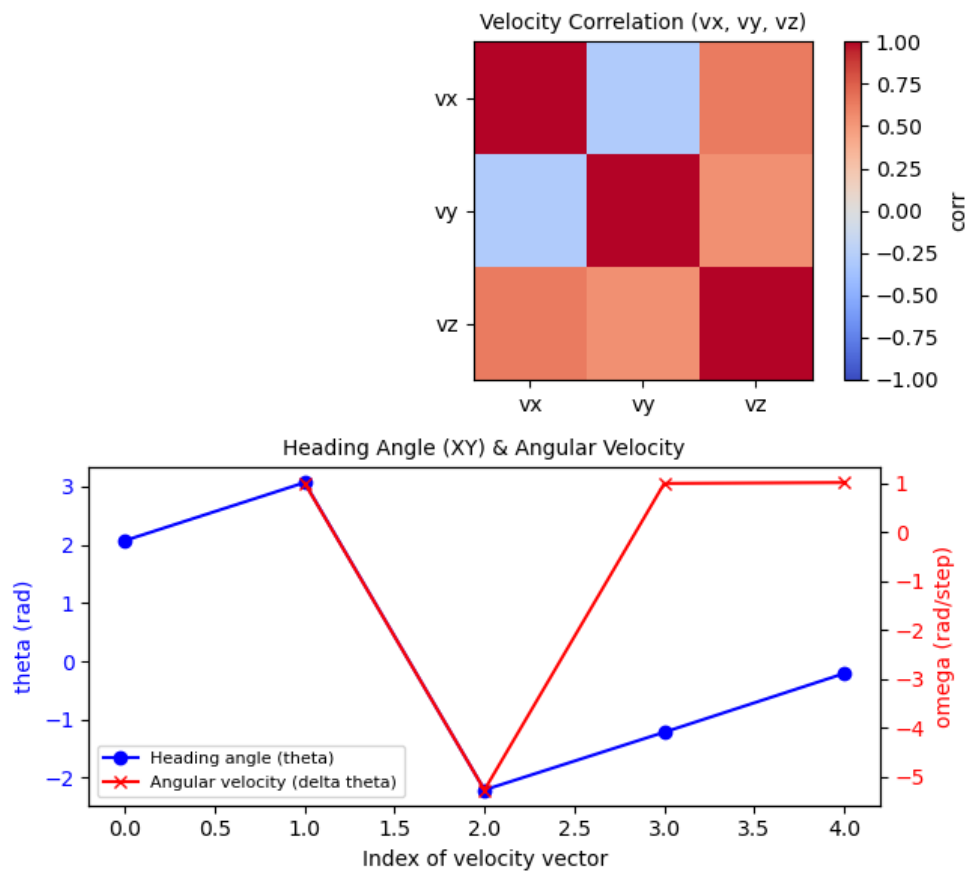


Figure 5: Velocity Correction

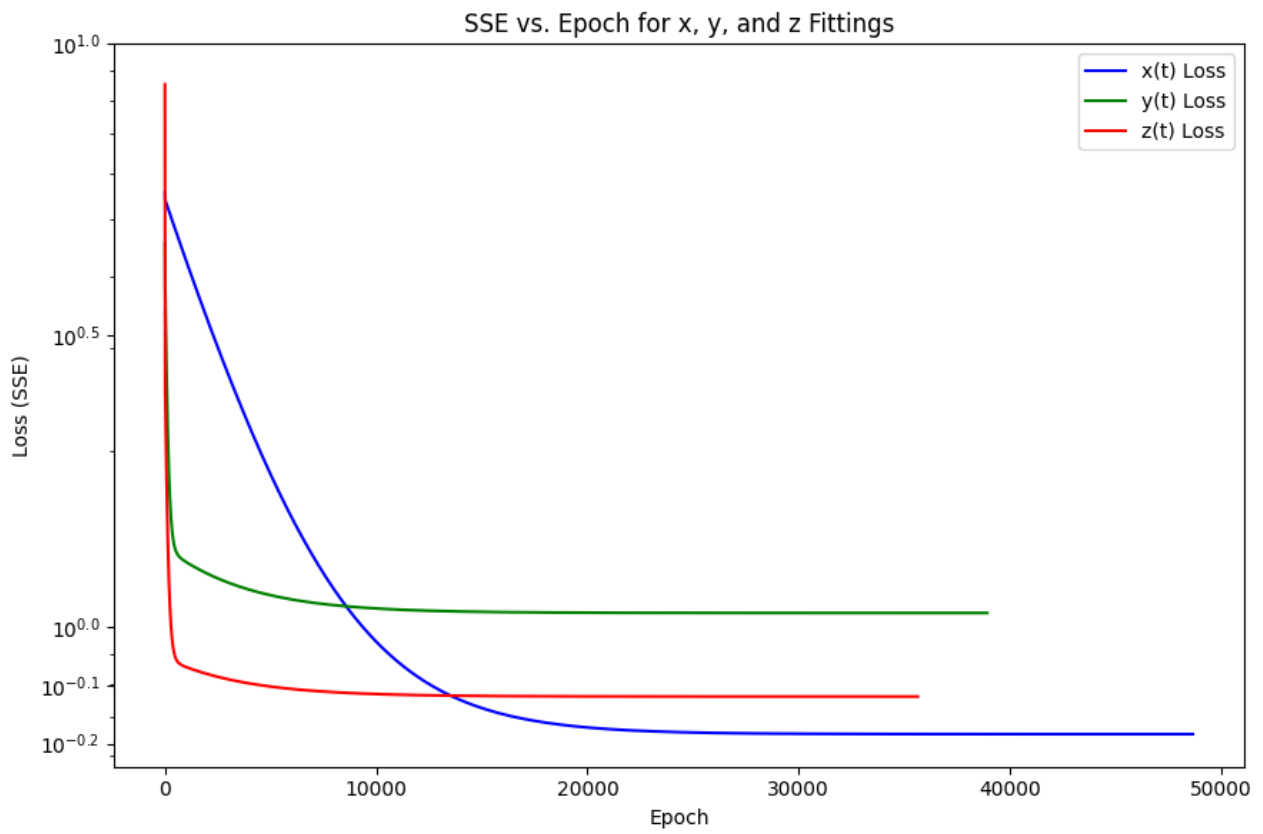


Figure 6: SSE and Epoch for x, y, and z Fittings

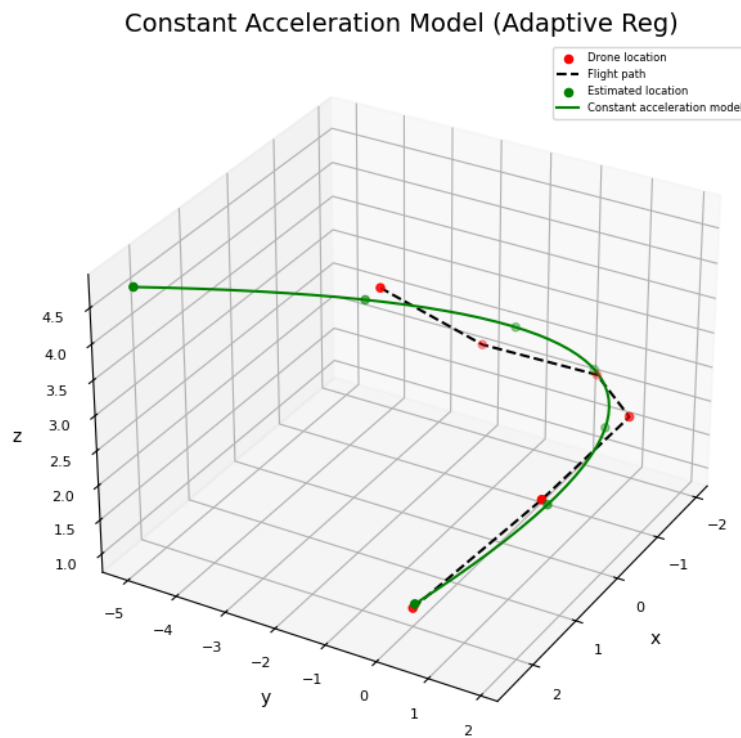


Figure 7: Constant Accelerate Model and the Actual Position (plus covariance and angular velocity regression)

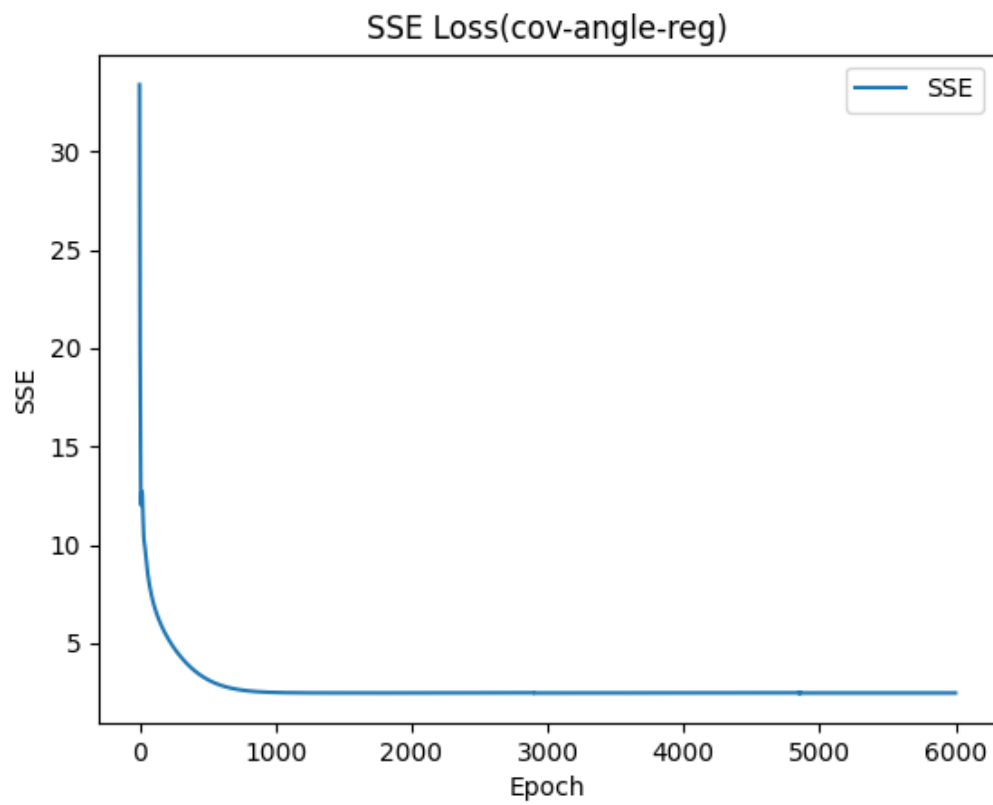


Figure 8: Constant Accelerate Model and the Actual Position(plus covariance and angular velocity regression)

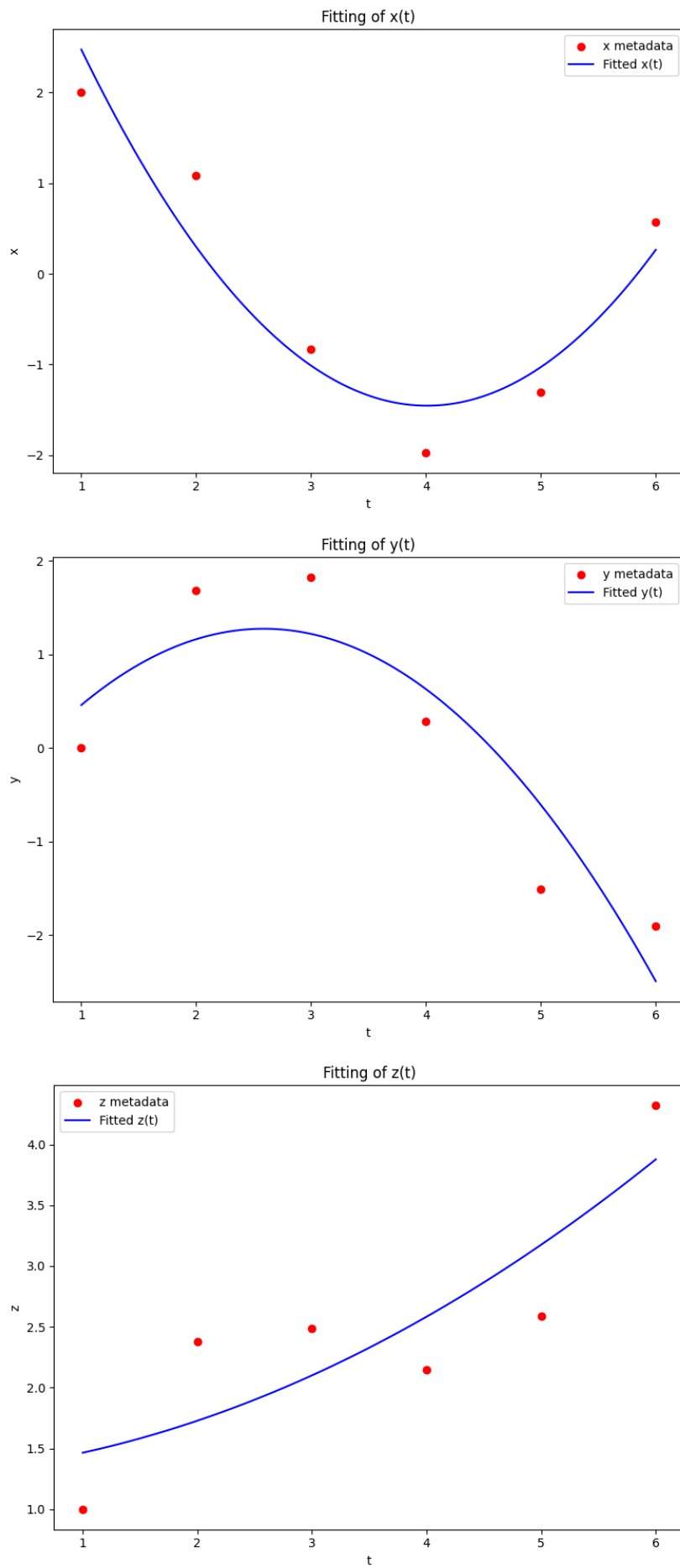


Figure 9: Fitting Results for x, y, and z

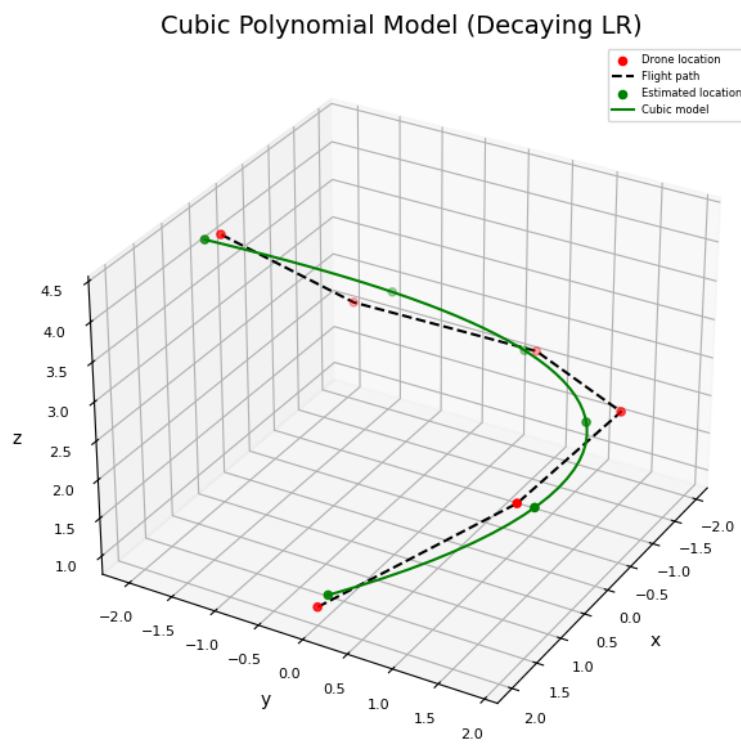


Figure 10: Cubic Model and the Actual Position

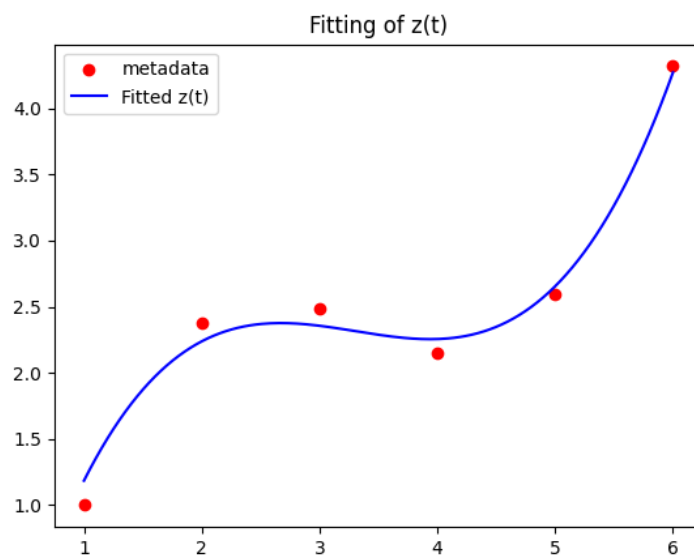
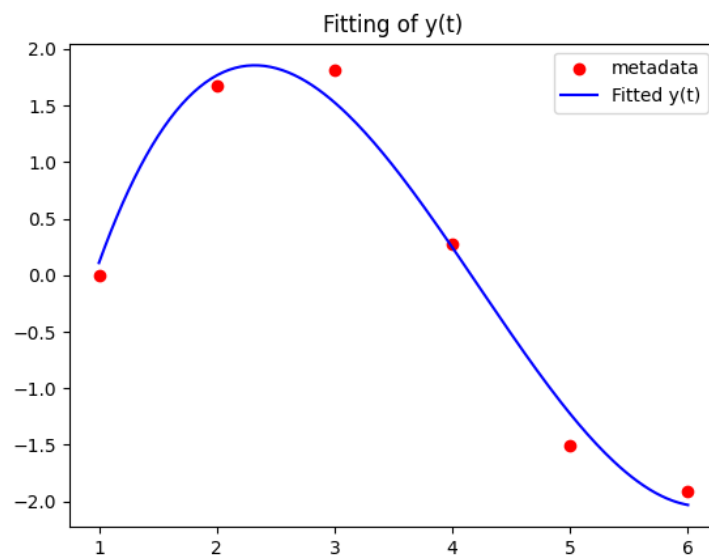
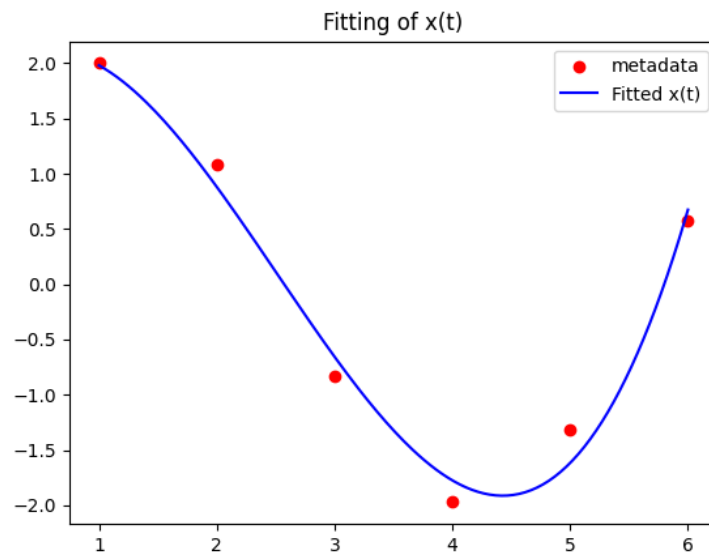


Figure 11: Fitting Results for x , y , and z